



Universidad Nacional Autónoma De México

Facultad de Ingeniería

Bitácora de IA

Presenta:

Aquino Martínez Alexis Emmanuel

Cortes Suarez Alexis

Pérez Cruz Emilio Alejandro

Verona Lara Alan Jahir

Profesor:

Ing. Ariel Adara Mercado Martínez

Materia:

Estructura y Programación de
Computadores.

Fecha:

11 de junio de 2026

Introducción

En el presente documento se detallará los diferentes usos que tuvo diferentes IA a lo largo de la planificación, creación e implementación del proyecto. Esto con el propósito de tener un registro de como y en que se utilizó la misma en diferentes procesos.

Etapa 1 — Configuración inicial del Repositorio

Responsable	Todos los integrantes
Herramienta IA	Gemini - (Google) y ChatGPT (OpenAI)

Análisis: Al entrar al Git-Hub de la asignatura la primera tarea fue estructurar el proyecto conforme a los directorios indicados en la rúbrica: src/, include/, tests/, docs/, scripts/ y examples/. A pesar de ya haber realizado un trabajo con anterioridad en Git-Hub se nos seguía dificultando la interfaz de esta.

Prompts: Usamos varios prompts, pero los más destacables fueron los siguientes.

- “¿Cómo puedo crear la estructura de carpetas indicada en la rúbrica directamente desde GitHub sin usar la terminal?”
- “GitHub no me deja subir carpetas vacías, ¿cómo le hago para que aparezcan en el repositorio?”
- “¿Cómo funciona lo de las ramas en GitHub? ¿Cada uno tiene que hacer una rama diferente o todos trabajamos en main?”

Contenido generado por la IA: Gemini explicó que Git no rastrea directorios vacíos por diseño, y orientó al equipo a colocar un archivo placeholder en cada carpeta para forzar su indexación. Además, nos ayudó a crear las carpetas, entender que eran los “pull request” y consejos para trabajar individualmente.

Errores encontrados: Al crear carpetas desde la interfaz web, algunas quedaban anidadas incorrectamente dentro de otras por errores en la selección del directorio activo. Se resolvió prestando atención al nivel de directorio visible antes de crear cada nueva carpeta.

Etapa 2 — Analizador Léxico

Responsable	Suarez Cortes Alexis
Herramienta IA	Gemini - (Google)

Análisis: El lexer es el inicio de todo. Lee el archivo .asm carácter por carácter y clasifica cada elemento en un tipo de token. Alexis partió del fragmento de código base provisto en el README del repositorio, que mostraba cómo abrir y leer un archivo con fopen y fgetc.

Prompts: Usamos varios prompts, pero los más destacables fueron los siguientes.

- *“Tenemos este código base del README para leer archivos. ¿Cómo lo modificamos en C para que extraiga tokens ignorando espacios y tabuladores?”*
- *“¿Cómo hago en C una función que verifique si una palabra es un registro válido de IA-32 como EAX, EBX, ECX, EDX, ESI, EDI, EBP o ESP?”*
- *“¿Por qué mi lexer se salta tokens cuando los operandos están pegados, como en MOV EAX,[EBX]?”*

Contenido generado por la IA: Gemini generó la estructura base del lexer: un bucle principal con fgetc que toma decisiones según el carácter actual, la función

is_register con arreglo de cadenas y strcmp, el manejo de ungetc para devolver caracteres al flujo cuando el token ya estaba completo.

Errores encontrados: No nos arrojó ningún error por el momento.

Etapa 3 — Analizador Sintáctico

Responsable	Suarez Cortes Alexis
Herramienta IA	Gemini - (Google)

Análisis: El parser recibe los tokens y los organiza en estructuras que representan instrucciones completas. Su tarea más compleja es identificar correctamente el modo de direccionamiento de cada operando, distinguiendo entre casos como [EBX], [EBP+4] o [EBX+ECX*4+8], que requieren la construcción del byte SIB.

Prompts: Usamos varios prompts, pero los más destacables fueron los siguientes.

- “Tomando como base los tokens que ya genera nuestro Lexer, ¿Cómo implementamos en C un parser que valide los modos de direccionamiento de IA-32?”
- “¿Cómo diseño las estructuras en C que conecten al parser con el encoder?”
- “Mi parser marca error en donde se ve la imagen, resuelve.”

Contenido generado por la IA: Gemini generó el esquema de las estructuras. También produjo la estructura base del parser y explicó el mecanismo de lookahead para distinguir la coma como separador de operandos de la coma como error dentro de una expresión de memoria.

Errores encontrados: Las expresiones de memoria con coma interna confundían al parser. Gemini reestructuró el flujo para que, al entrar al modo de parseo de memoria tras detectar el corchete de apertura

Etapa 4 — Backend del Ensamblador

Responsable	Verona Lara Alan Jahir
Herramienta IA	Gemini (Google)

Análisis: El backend implementa la tabla de símbolos, la primera pasada que registra etiquetas y calcula el contador de localización, y la segunda pasada que genera el código máquina y aplica fixups a referencias adelantadas. El integrante trabajó con sus propias versiones del lexer y parser integradas en el mismo módulo para poder hacer pruebas de forma independiente.

Prompts: Usamos varios prompts, pero los más destacables fueron los siguientes.

- *¿Cómo implemento una tabla de símbolos en C que soporte referencias adelantadas?*
- *¿Cómo funciona los fixups en un ensamblador de dos pasadas?*
- *¿Cuál es la función de que haga dos pasadas? ¿Está bien si solo hace una?*
- *Solucionar error: VS Code no reconoce GCC en Windows con MSYS2. El compilador está en C:/msys64/ucrt64/bin/gcc.exe.*

Contenido generado por la IA: Gemini generó symtab.h con la definición de Symbol y los prototipos, symtab.c con la implementación completa de las cuatro funciones de la tabla, la lógica de la segunda pasada en parser.c incluyendo la

generación del archivo output.hex con los bytes calculados, y el main.c que orquesta ambas pasadas

Errores encontrados: El ejecutable generado era eliminado por el sistema operativo por configuraciones de seguridad de Windows. Se resolvió configurando exclusiones de carpeta.

Etapa 5 — Encoder IA-32

Responsable	Perez Cruz Emilio Alejandro
Herramienta IA	Claude (Anthropic)

Análisis: El encoder traduce las instrucciones representadas por el parser a secuencias de bytes de código máquina IA-32. Es el módulo más técnico del proyecto, ya que requiere conocimiento preciso del formato de instrucciones x86: opcodes, bytes ModRM, bytes SIB, desplazamientos en little-endian e inmediatos

Prompts: Usamos varios prompts, pero los más destacables

- *Genera encoder_encode_mov para las variantes de MOV en IA-32.*
- *Escribe encoder_encode_alu cubriendo ADD, SUB, AND, OR, XOR y CMP*
- *Genera encoder_encode_unary para INC, DEC, NOT, NEG, MUL, DIV, PUSH y POP.*
- *Necesito encoder_encode_jump y encoder_encode_call. Los saltos son relativos a la instrucción siguiente: disp32 = destino - (dirección actual + tamaño instrucción).*

Contenido generado por la IA: Claude generó el contenido completo de `encoder.c` en bloques sucesivos: las primitivas `encoder_build_modrm`, `encoder_build_sib`, `encoder_write_le32` y `encoder_write_le8`; el codificador de operandos de memoria `encoder_encode_mem_operand` con todos los casos de direccionamiento incluyendo los casos especiales de ESP y EBP; y las cinco funciones de codificación por familia

Errores Encontrados: Al intentar integrar el encoder con el backend del integrante 3 se detectó que ambos habían definido versiones ligeramente distintas de la estructura `Operand`. Se resolvió consolidando la definición en `ia32_types.h` como fuente única.

Etapa 6 — Formato Objeto y Linker

Responsable	Aquino Martínez Alexis Emmanuel
Herramienta IA	Deepseek – (High-Flyer) , ChatGPT (OpenAI) y Claude (Anthropic)

Análisis: El módulo de formato objeto y linker es el último eslabón del pipeline. Toma los bytes generados por el encoder, los empaqueta en un archivo objeto con encabezado, secciones, tabla de símbolos y tabla de relocalaciones, y posteriormente combina múltiples archivos objeto en un binario ejecutable final.

Prompts: Usamos varios prompts, pero los más destacables fueron los siguientes.

- “Explícame cómo funciona un archivo objeto: qué partes tiene, qué es una relocalación y cómo funciona el linker en términos generales
- Diseña `object.h` en C con las estructuras `ObjHeader`, `Section`, `Symbol`, `Relocation` y `ObjectFile`. Incluye soporte para símbolos locales, globales y externos, y relocalaciones de tipo `REL32` y `ABS32`.

- Diseña linker.h con la estructura LinkerCtx y los prototipos de las cuatro etapas del linker: fusión de secciones, construcción de tabla global de símbolos, aplicación de relocalaciones y escritura del binario final.
- Escribe tests/test_linker.c con tres casos: objeto básico con escritura y lectura, dos módulos enlazados con CALL entre ellos usando relocalación REL32, y detección de símbolo externo no resuelto.

Contenido generado por la IA: Se generoo parte de los diferentes módulos con sus respectivas estructuras y funciones.

Errores encontrados: Al trabajar directamente en Git-Hub no fue posible compilar y ejecutar el código durante el desarrollo, si no hasta las partes finales.
